Flying into the Quantum Era: Secure UAV Communication with Expert Policy Scheduling

[Author Names Redacted for Review]
[Affiliation Redacted for Review]

***Abstract*—Unmanned Aerial Vehicles (UAVs) have become indispensable across civilian and defence domains, yet the MAVLink protocol that governs their command-and-control communication was designed without cryptographic protection, leaving telemetry and flight commands vulnerable to eavesdropping, injection, and replay attacks. The anticipated arrival of cryptographically relevant quantum computers further threatens classical public-key schemes such as RSA and ECDH through Shor's algorithm, rendering any "store-now, decrypt-later" capture immediately actionable. We present SECURETUNNEL, a complete, deployment-ready post-quantum cryptographic tunnel that transparently secures MAVLink traffic between a drone companion computer and a ground control station. The system implements a 72-suite registry formed by the level-consistent Cartesian product of nine key-encapsulation mechanisms (ML-KEM, HQC, Classic McEliece at NIST levels 1, 3, and 5), eight digital signature schemes (ML-DSA, Falcon, SPHINCS+), and three authenticated encryption with associated data ciphers (AES-256-GCM, ChaCha20-Poly1305, Ascon-128a). A two-message post-quantum handshake establishes session keys via KEM encapsulation, authenticates the ground station through digital signatures, and verifies the drone with HMAC-based pre-shared key authentication. We evaluate all 72 suites on a Raspberry Pi 4 companion computer with a Pixhawk flight controller, measuring handshake latency, per-packet AEAD cost, INA219-sampled power consumption at $1\,\mathrm{kHz}$, CPU temperature, and live MAVLink telemetry integrity across three operational phases: baseline, XGBoost-based DDoS detection, and Transformer-based DDoS detection. Results show that ML-KEM-768 with ML-DSA-65 completes a full post-quantum handshake in $21.3\,\mathrm{ms}$ at $3.91\,\mathrm{W}$, while the fastest suite (ML-KEM-512 with Falcon-512) achieves $11.1\,\mathrm{ms}$. A novel three-axis expert policy scheduler (MDEAS) jointly optimises AEAD selection, NIST security level, and DDoS detection intensity based on real-time power, temperature, link quality, and battery state, enabling graceful degradation under resource constraints. Our contributions span the first comprehensive PQC suite registry for UAVs, a measurement-driven cost model with 72 characterised suites, and an energy-aware scheduling architecture deployable on resource-constrained embedded platforms.**

***Index Terms*—Post-Quantum Cryptography, UAV Security, MAVLink, Key Encapsulation Mechanism, Digital Signature, Authenticated Encryption, Embedded Systems, DDoS Detection, Crypto Agility**

I. INTRODUCTION

Unmanned Aerial Vehicles have transitioned from niche military platforms to ubiquitous tools deployed in precision agriculture, infrastructure inspection, emergency response, and logistics [27]. The global commercial drone market is projected to exceed USD 54 billion by 2030, with millions of autonomous systems sharing regulated and unregulated airspace. At the core of most small UAV systems lies *MAVLink* [29], a lightweight binary telemetry protocol that carries flight commands, sensor data, and heartbeat signals between a flight controller and a ground control station (GCS). MAVLink v2 introduced message signing with SHA-256 HMAC, yet it lacks key establishment, forward secrecy, and confidentiality—any adversary with network proximity can observe, inject, or replay commands [28].

This vulnerability is compounded by the looming quantum threat. Shor's algorithm [16] breaks RSA, DSA, and elliptic-curve Diffie–Hellman in polynomial time on a sufficiently large quantum computer; Grover's algorithm [17] halves the effective security of symmetric ciphers. The National Institute of Standards and Technology (NIST) has responded by standardising post-quantum algorithms: ML-KEM (FIPS 203) [1] for key encapsulation, ML-DSA (FIPS 204) [2] for digital signatures, and SLH-DSA (FIPS 205) [3] as a hash-based signature alternative. Additional candidates—HQC [9] (code-based) and Classic McEliece [10] (code-based, conservative)—remain under evaluation in NIST's fourth round [4]. Despite this standardisation progress, *no prior system integrates a comprehensive PQC suite registry, real-time cost measurement, and adaptive scheduling into a deployable UAV tunnel*.

A. Problem Statement and Research Gap

Existing PQC benchmarks for embedded platforms [22], [23] focus on individual primitive timing in isolation, without measuring the compound cost of a complete handshake over a live MAVLink session. PQC tunnel prototypes such as Rosenpass [25] target VPN links but do not address UAV-specific constraints: limited power budgets, thermal throttling, real-time telemetry deadlines, and the need for crypto agility during flight. Meanwhile, drone security research [27], [28] identifies the threat but stops short of a full system implementation with measured deployment cost.

B. Contributions

This paper makes the following contributions:

C1 PQC Suite Registry. A 72-suite registry constructed from the NIST-level-consistent Cartesian product of 9 KEMs $\times$ 8 signatures $\times$ 3 AEADs, with a two-message handshake protocol providing forward secrecy, mutual authentication, and anti-downgrade protection.

C2 Measurement-Driven Cost Model. A comprehensive benchmark of all 72 suites on a Raspberry Pi 4 companion computer with Pixhawk flight controller, measuring handshake latency, per-packet AEAD cost, INA219 power at 1 kHz, temperature, and MAVLink integrity across baseline and two DDoS-detection configurations.

C3 Three-Axis Expert Scheduler (MDEAS). A novel Measurement-Driven Energy-Aware Scheduler that jointly optimises AEAD selection (data-plane axis), NIST security level (control-plane axis), and DDoS detection intensity (compute-plane axis) using real-time telemetry, thermal prediction, and break-even cost analysis.

C4 Deployable System. An open-source, end-to-end implementation operating as a transparent "bump-in-the-wire" proxy requiring no firmware modification, validated with live ArduPilot telemetry over WiFi.

C. Paper Organisation

Section II surveys related work. Section III defines the threat model. Section IV describes the post-quantum system architecture. Section V details the experimental setup. Section VI presents results and analysis. Section VII discusses trade-offs and limitations. Section VIII concludes with future directions.

II. Related Work

A. Post-Quantum Cryptography for Embedded Systems

The pqm4 project [22] provides optimised ARM Cortex-M4 implementations of NIST PQC candidates, establishing micro-benchmark baselines for key generation, encapsulation, and signing on bare-metal platforms. Kannwischer et al. [23] extended this to Cortex-A processors, reporting ML-KEM-768 key generation in 0.33 ms on a Cortex-A72 at 1.8 GHz. These works benchmark primitives in isolation; they do not measure compound handshake cost over a live protocol session, nor do they account for the energy and thermal cost of sustained operation. Our work measures all 72 suites end-to-end on a Raspberry Pi 4 (Cortex-A72) with INA219 power instrumentation, capturing the true deployment cost including MAVLink overhead.

B. PQC in Network Protocols

Post-quantum migration has been studied in TLS 1.3, where Paquin et al. [24] evaluated hybrid key exchange using liboqs [19]. Rosenpass [25] adds a PQC handshake layer atop WireGuard, using Classic McEliece for long-term keys and Kyber for ephemeral exchange. Hülsing et al. [26] analysed PQC overhead in IPsec and OpenVPN deployments. These protocol-level studies target server-class hardware with ample resources. In contrast, our tunnel targets a 4 W embedded companion computer with thermal constraints and battery awareness, requiring an adaptive scheduling layer absent in prior protocol work.

The Open Quantum Safe (OQS) project [19], [20] provides the `liboqs` C library and its Python wrapper, implementing all NIST PQC finalists and additional candidates. We build directly on `liboqs-python` for all KEM and signature operations, using version 0.14.0 with ARM NEON optimisations enabled.

C. UAV Security and MAVLink Protection

Altawy and Youssef [27] surveyed security, privacy, and safety aspects of civilian drones, identifying MAVLink's lack of encryption as a critical vulnerability. Pham and Vakilinia [28] analysed post-quantum cryptography for UAV communication, evaluating KEM and signature overhead on ARM processors but without a complete tunnel implementation or runtime adaptation. The MAVLink specification [29] defines optional message signing (SHA-256 HMAC) in version 2.0, but this provides only integrity, not confidentiality or forward secrecy. Our system provides a *complete* transparent tunnel: KEM-based key establishment, signature-verified authentication, AEAD-encrypted data plane, and runtime suite adaptation—all without modifying the flight controller firmware.

D. DDoS Detection on Edge Devices

Lightweight DDoS detection on embedded platforms has been explored using ensemble methods (XGBoost, Random Forest) and deep learning architectures. Sharafaldin et al. [34] introduced the CIC-IDS dataset widely used for intrusion detection benchmarking. More recent work has applied Transformer-based models to network traffic classification [35], achieving higher accuracy at a significant computational cost. Our system integrates two detector tiers: an XGBoost model with 54 features and ≈7 ms inference time, and a Time Series Transformer (TST) model with 46 features and ≈11 ms inference time. Critically, we measure the *system-level impact* of running these detectors concurrently with PQC operations: XGBoost adds 0.95 W and 4.8 °C; TST adds 1.97 W and 10.7 °C—costs that must be budgeted by the scheduler.

E. Crypto Agility and Runtime Adaptation

Crypto agility—the ability to switch algorithms at runtime—is recognised as essential for post-quantum migration [4]. However, most systems treat it as a configuration-time choice rather than a runtime decision. Barker [36] recommends algorithm transitions but does not address the dynamic context of resource-constrained flight. Our MDEAS scheduler treats crypto agility as a *continuous optimisation problem* with three axes (AEAD, security level, detector intensity), using hysteresis, break-even analysis, and thermal prediction to avoid oscillation while responding to changing conditions.

III. Threat Model

We consider four adversary classes with increasing capability, following the Dolev–Yao model [18] extended with quantum computation.

A. Adversary Classes

$\mathcal{A}_1$**: Passive Eavesdropper.** Captures all network traffic on the wireless link between drone and GCS. Can perform "store now, decrypt later" for future quantum decryption.

$\mathcal{A}_2$**: Active Network Adversary.** Controls the network channel (Dolev–Yao model): can intercept, modify, replay, delay, and inject packets. Can mount man-in-the-middle attacks, injection of rogue MAVLink commands, and GPS spoofing via crafted `GLOBAL_POSITION_INT` messages.

$\mathcal{A}_3$**: DDoS Adversary.** Floods the encrypted channel with malformed or replayed packets to exhaust computational resources on the companion computer, degrading real-time telemetry delivery.

$\mathcal{A}_4$**: Quantum Adversary.** Possesses a Cryptographically Relevant Quantum Computer (CRQC) capable of running Shor's algorithm against RSA, DSA, and ECDH, and Grover's algorithm to accelerate brute-force search against symmetric ciphers.

B. Trust Assumptions

- **T1** The flight controller firmware (ArduPilot [31]) and companion OS are trusted.
- **T2** A pre-shared key (PSK) is securely provisioned on the drone before deployment.
- **T3** The GCS signing key pair is generated offline and the public key is distributed to the drone.
- **T4** The `liboqs` [19] library correctly implements the NIST PQC algorithms.
- **T5** Physical access to the companion computer during flight is infeasible.

C. Security Goals

- **G1 Confidentiality.** All MAVLink traffic between drone and GCS is encrypted with AEAD, rendering eavesdropping (including quantum-assisted "store now, decrypt later") ineffective.
- **G2 Integrity and Authenticity.** Every packet carries an AEAD authentication tag bound to the 22-byte header as associated data; any modification is detected.
- **G3 Replay Protection.** A sliding-window replay filter (default window $W = 1024$, following RFC 6479 / WireGuard pattern) rejects duplicate sequence numbers.
- **G4 Forward Secrecy.** Ephemeral KEM key pairs are generated per session; compromise of long-term keys does not reveal past session keys.
- **G5 Mutual Authentication.** The GCS proves its identity via digital signature; the drone proves its identity via HMAC over PSK.
- **G6 Anti-Downgrade.** The wire version byte is included in the signed transcript, preventing protocol version rollback.
- **G7 Availability under DDoS.** Integrated ML-based detection identifies volumetric attacks; the scheduler downgrades to lighter suites to maintain telemetry throughput.

Table I maps each adversary to the system mechanisms that counter it.

TABLE I: Threat–Mitigation Mapping

Adversary	Countermeasure
$\mathcal{A}_1$ (Eavesdrop)	AEAD encryption (G1), PQ KEM key exchange
$\mathcal{A}_2$ (Active)	Digital signature auth (G5), AEAD integrity (G2), replay window (G3), anti-downgrade (G6)
$\mathcal{A}_3$ (DDoS)	ML detectors, scheduler degradation (G7), rate-limited handshake
$\mathcal{A}_4$ (Quantum)	NIST PQC KEMs and signatures (G1, G4, G5)

[Figure Placeholder]

System architecture showing drone-side (Pixhawk → MAVProxy → PQC Proxy) and GCS-side (PQC Proxy → MAVProxy → QGC) with encrypted UDP channel, TCP control plane, and optional DDoS detector.

Fig. 1: End-to-end system architecture of SECURETUNNEL.

IV. POST-QUANTUM SYSTEM ARCHITECTURE

A. System Overview

SECURETUNNEL operates as a transparent "bump-in-the-wire" proxy that requires no modification to the flight controller firmware or ground station software. Fig. 1 illustrates the system topology.

The drone-side companion computer (Raspberry Pi 4) runs MAVProxy [30] connected to the Pixhawk flight controller [33] via USB serial. MAVProxy forwards telemetry to the PQC proxy over local UDP (ports 47003/47004). The proxy encrypts each packet with the negotiated AEAD cipher using the session key derived during the PQC handshake, transmits it over WiFi on the encrypted UDP channel (ports 46011/46012), and the GCS-side proxy decrypts and delivers to the ground MAVProxy instance. A TCP control channel (port 48080) carries scheduler commands, clock synchronisation, and rekey negotiation between the drone-driven scheduler and the GCS follower.

B. Suite Registry Design

The suite registry is the combinatorial backbone of SECURETUNNEL. Each suite is a triple $(\mathrm{KEM}, \mathrm{SIG}, \mathrm{AEAD})$ where the KEM and SIG must share the same NIST security level. We include algorithms from three KEM families and three signature families at three NIST levels, crossed with three AEAD ciphers.

NIST Level Mapping.

- **L1** (≈AES-128 security): ML-KEM-512, HQC-128, Classic-McEliece-348864, ML-DSA-44[1], Falcon-512, SPHINCS$^+$-128s.

[1]ML-DSA-44 is classified as NIST Level 2 in FIPS 204; we map it to L1 for practical pairing with L1 KEMs following the rationale that L2 exceeds L1 by a small margin.

TABLE II: KEM Algorithm Parameters (from liboqs 0.14.0)

Algorithm	Level	PK (B)	CT (B)	SS (B)
ML-KEM-512	L1	800	768	32
ML-KEM-768	L3	1 184	1 088	32
ML-KEM-1024	L5	1 568	1 568	32
HQC-128	L1	2 249	4 433	64
HQC-192	L3	4 522	9 026	64
HQC-256	L5	7 245	14 469	64
McEliece-348864	L1	261 120	96	32
McEliece-460896	L3	524 160	156	32
McEliece-8192128	L5	1 357 824	208	32

TABLE III: Signature Algorithm Parameters (from liboqs 0.14.0)

Algorithm	Level	PK (B)	SK (B)	Sig (B)
ML-DSA-44	L1	1 312	2 560	2 420
ML-DSA-65	L3	1 952	4 032	3 309
ML-DSA-87	L5	2 592	4 896	4 627
Falcon-512	L1	897	1 281	$\leq$666
Falcon-1024	L5	1 793	2 305	$\leq$1 280
SPHINCS$^+$-128s	L1	32	64	7 856
SPHINCS$^+$-192s	L3	48	96	16 224
SPHINCS$^+$-256s	L5	64	128	29 792

- **L3** ($\approx$AES-192 security): ML-KEM-768, HQC-192, Classic-McEliece-460896, ML-DSA-65, SPHINCS$^+$-192s.
- **L5** ($\approx$AES-256 security): ML-KEM-1024, HQC-256, Classic-McEliece-8192128, ML-DSA-87, Falcon-1024, SPHINCS$^+$-256s.

The level-consistent pairing produces:

$$|\mathcal{S}| = \underbrace{(3 \times 3)}_{\text{L1}} + \underbrace{(3 \times 2)}_{\text{L3}} + \underbrace{(3 \times 3)}_{\text{L5}} = 24 \tag{1}$$

KEM$\times$SIG pairs (L3 has no Falcon variant), each crossed with 3 AEADs:

$$N_{\text{suites}} = 24 \times 3 = 72 \tag{2}$$

Each suite is assigned a canonical identifier of the form `cs-{kem}-{aead}-{sig}` (e.g., `cs-mlkem768-aesgcm-mldsa65`).

Table II summarises the KEM parameters, and Table III summarises the signature parameters.

C. KEM-Based Key Establishment

The handshake uses a two-message protocol over TCP (port 46000):

Message 1: ServerHello (GCS $\rightarrow$ Drone). The GCS generates an ephemeral KEM key pair $(pk_{\text{kem}}, sk_{\text{kem}}) \leftarrow \text{KEM.KeyGen}()$, a random 8-byte session identifier sid, and a random 8-byte challenge c. It constructs the transcript:

$$T = v \| \texttt{pq-drone-gcs:v1} \| sid \| \text{kem\_name} \| \text{sig\_name} \| pk_{\text{kem}} \| c \tag{3}$$

where v is the frozen wire version byte. The GCS signs the transcript: $\sigma_{\text{gcs}} \leftarrow \text{SIG.Sign}(sk_{\text{gcs}}, T)$ and sends the length-prefixed wire message containing all fields plus σ_{gcs}.

Message 2: ClientReply (Drone $\rightarrow$ GCS). The drone verifies σ_{gcs} using the pre-distributed GCS public key, checks that the negotiated algorithms match the expected suite (anti-downgrade), and computes:

$$(ct, K) \leftarrow \text{KEM.Encaps}(pk_{\text{kem}}) \tag{4}$$

where K is the shared secret. It also computes an HMAC authentication tag over the entire ServerHello wire:

$$\tau = \text{HMAC-SHA256}(PSK, ServerHello_{\text{wire}}) \tag{5}$$

and sends (ct, τ).

Key Derivation. Both sides derive directional transport keys using HKDF-SHA256 [12]:

$$(k_{d \to g} \| k_{g \to d}) = \text{HKDF}(K, \text{“pq-drone-gcs|hkdf|v1”}, info, 64) \tag{6}$$

where $info$ incorporates sid, kem_name, and sig_name. The shared secret K is zeroed immediately after derivation, following the WireGuard pattern.

D. AEAD Data-Plane Framing

Each plaintext MAVLink packet is encrypted under the negotiated AEAD cipher. The wire format is:

$$frame = \underbrace{H}_{\text{22 B header}} \| \text{AEAD.Enc}(k, iv, pt, H) \tag{7}$$

The 22-byte header H is structured as:

$$H = (v, kem\_id, kem\_p, sig\_id, sig\_p, sid^8, seq^8, epoch)$$

and is bound as associated authenticated data (AAD) to the AEAD operation. The 12-byte initialisation vector is deterministically constructed from the epoch byte and sequence number, avoiding IV transmission and saving 12 bytes per packet:

$$iv = epoch \parallel seq_{11\text{B}} \tag{8}$$

Replay protection follows the WireGuard/RFC 6479 sliding-window approach with a configurable window W (default 1024). The implementation uses a split *check/commit* pattern: the replay window is checked *before* AEAD decryption and committed *after* successful authentication, preventing a malicious packet from shifting the window without valid authentication.

E. Rekey Protocol

Two rekey modes are supported:

Cold Restart. The proxy process is terminated, a new handshake is performed with the target suite, and a fresh proxy is started. This mode provides the strongest isolation between suites, as no key material persists across the restart boundary. It incurs a brief blackout ($\sim$2 s) during transition.

In-Band Rekey. The proxy remains running; the scheduler sends a `prepare_rekey` control message (packet type 0x02) through the encrypted channel. Both sides execute a new handshake on the TCP port (serialised with a lock to avoid port conflicts), atomically swap the AEAD `Sender`/`Receiver` objects, and zero the old key material. This minimises blackout

but requires careful coordination via a two-phase commit protocol:

1) Coordinator → Follower: `prepare_rekey(`*suite*`,` *rid*`)`
2) Follower → Coordinator: `prepare_ok(`*rid*`)` or `prepare_fail`
3) Coordinator → Follower: `commit_rekey(`*suite*`,` *rid*`)`
4) Both execute handshake → atomic cipher swap

The rekey overhead fraction for a session of duration R with handshake time T_{hs} is:

$$\Phi(R) = \frac{T_{\text{hs}}}{R + T_{\text{hs}}} \tag{9}$$

For the default suite (ML-KEM-768 + ML-DSA-65, $T_{\text{hs}} = 21.3\,\text{ms}$) with a 60-second rekey interval, $\Phi = 0.035\%$.

F. DDoS Detection Pipeline

The DDoS detection pipeline runs on the companion computer alongside the PQC proxy. Two detection tiers are available:

XGBoost Detector. A gradient-boosted tree model operating on 54 extracted features per traffic window. Inference takes ∼7 ms per window, consuming an additional 0.95 W and raising temperature by $4.8\,^{\circ}\text{C}$.

Time Series Transformer (TST) Detector. A Transformer-based intrusion detection model operating on 46 features. Inference takes ∼11 ms per window, consuming an additional 1.97 W and raising temperature by $10.7\,^{\circ}\text{C}$.

Only one detector runs at any time (the companion computer lacks resources for concurrent operation). The scheduler manages transitions between detector levels using the same process-group lifecycle management as proxy processes.

G. Three-Axis Expert Scheduler (MDEAS)

The Measurement-Driven Energy-Aware Scheduler (MDEAS) treats crypto agility as a joint optimisation problem over three axes:

Axis 1 — AEAD Selection (data-plane, per-packet cost). Selects among {ChaCha20-Poly1305, AES-256-GCM, Ascon-128a} based on measured per-packet encrypt/decrypt latency, temperature, and CPU utilisation. The AEAD cost profile is maintained as an exponentially weighted moving average (EWMA) with $\alpha = 0.2$ (steady state) and pre-seeded from benchmark data (Benchmark-Seeded Initialisation):

$$\hat{C}_{n+1} = \alpha \cdot C_{\text{obs}} + (1 - \alpha) \cdot \hat{C}_n \tag{10}$$

Axis 2 — NIST Security Level (control-plane, per-handshake cost). Selects among {L1, L3, L5} with a fixed KEM+SIG pairing per level: L1→(ML-KEM-512, ML-DSA-44), L3→(ML-KEM-768, ML-DSA-65), L5→(ML-KEM-1024, ML-DSA-87).

Axis 3 — DDoS Detection Level (compute-plane, background cost). Selects among {None, XGBoost, TST} based on threat indicators with awareness of detector overhead.

Decision Cascade. The scheduler evaluates a 12-gate priority cascade at each tick:

1. Telemetry stale ($> 2\,\text{s}$) → HOLD
2. DDoS detected (drop ratio > 0.10) → EMERGENCY
3. Battery critical ($< 14.0\,\text{V}$) → EMERGENCY
4. Temperature critical ($> 80\,^{\circ}\text{C}$) → EMERGENCY
5. Emergency active → kill detector
6. Detector thermal ceiling exceeded → DOWNGRADE_DETECTOR
7. Predictive thermal crossing (60 s extrapolation) → SWITCH_AEAD
8. Thermal/CPU stress → SWITCH_AEAD (with break-even)
9. AEAD recovery (stress subsided) → SWITCH_AEAD
10. Link degraded ($\text{gap}_{p95} > 1\,\text{s}$) → DOWNGRADE_LEVEL
11. Stable conditions, proactive rekey due → REKEY
12. Stable, upgrade hysteresis satisfied → UPGRADE

Break-Even Analysis. Before triggering an AEAD switch, the scheduler computes the break-even time:

$$t_{\text{be}} = \frac{C_{\text{rekey}}}{(\hat{C}_{\text{cur}} - \hat{C}_{\text{tgt}}) \cdot r_{\text{pkt}} \cdot 2} \tag{11}$$

where $C_{\text{rekey}} = 300\,\text{ms}$ is the handshake cost, $\hat{C}$ are the EWMA AEAD costs, and r_{pkt} is the packet rate. The switch is executed only if $t_{\text{be}} < 30\,\text{s}$ (nominal) or $t_{\text{be}} < 15\,\text{s}$ (under stress).

Hysteresis. Downgrade requires 5 s of sustained adverse conditions; upgrade requires 30 s of nominal conditions (tripled to 90 s when the vehicle is armed). This prevents oscillation during transient load spikes.

Cross-Axis Constraints. Not all axis combinations are permitted:

- Ascon-128a is forbidden with TST (combined overhead exceeds thermal budget).
- Classic McEliece is discouraged with TST (handshake length competes for CPU).
- Classic McEliece + SPHINCS$^+$ is forbidden at any detector level.

V. Experimental Setup

A. Hardware Platform

Drone Companion Computer. Raspberry Pi 4 Model B Rev 1.5 with Broadcom BCM2711 SoC (4-core ARM Cortex-A72 at 1.8 GHz), 4 GB LPDDR4 RAM. The CPU governor is locked to `performance` mode to eliminate frequency scaling artefacts. Power is measured using a Texas Instruments INA219 [32] bidirectional current/power monitor on the I^{2}C bus at address `0x40`, sampling at 1 kHz in high-speed ADC mode (BADC=0x0080, settle time 0.4 ms). The shunt resistor is $0.1\,\Omega$. Voltage is read from register 0x02 (bits [15:3], LSB = 4 mV); shunt voltage from register 0x01 (signed 16-bit, LSB = 10 µV). Energy is computed via trapezoidal integration over the power samples. Temperature is read from `/sys/class/thermal/thermal_zone0/temp`.

Flight Controller. Pixhawk 6C Mini [33] running ArduPilot [31] (ArduCopter v4.5.7), connected to the companion computer via USB serial. The flight controller provides real-time MAVLink telemetry including heartbeat, system status, battery voltage/current, attitude, and GPS position.

Ground Control Station. Windows laptop running MAVProxy and QGroundControl, connected to the drone over WiFi.

B. Software Stack

Cryptographic Libraries. `liboqs` [19] version 0.14.0 (C library with ARM NEON optimisations), accessed via `liboqs-python` [20]. AEAD ciphers use the `cryptography` [21] Python package (AES-GCM, ChaCha20-Poly1305) and the `ascon` [11] Python package (Ascon-128a).

Protocol Stack. MAVProxy [30] for MAVLink routing; `pymavlink` for protocol parsing and sequence tracking; `psutil` for system metrics; `smbus2` for INA219 register access.

DDoS Detectors. XGBoost model ($\sim$33 MB, 54 features) and TST model (Transformer-based, 46 features), both running in dedicated Python virtual environments on the companion computer.

Scheduling. The benchmarks use the `BenchmarkPolicy` with 10 s per suite, cold-restart mode, and sequential cycling through all 72 suites sorted by NIST level, then KEM, then signature.

C. Benchmark Orchestration

The benchmark is orchestrated by a PowerShell script (`run_e2e_benchmark.ps1`) that executes three phases:

1) **Phase 1 (Baseline):** All 72 suites with no DDoS detector.
2) **Phase 2 (XGBoost):** All 72 suites with XGBoost detector active.
3) **Phase 3 (TST):** All 72 suites with TST detector active.

Each phase proceeds as follows: the GCS benchmark server starts locally; the drone benchmark scheduler is launched via SSH; for each suite, the drone orchestrator sends a `start_proxy` command to the GCS, starts the local proxy, waits for the PQC handshake to complete, runs the MAVLink session for 10 s, collects metrics from both sides, and advances to the next suite. Clock synchronisation between drone and GCS is performed every 10 suites using a Chronos-style NTP exchange over the TCP control channel.

A 120 s inter-phase cool-down allows thermal recovery between phases.

D. Metrics Collection

For each suite, a comprehensive JSON document is produced containing 18 metric categories (detailed in Section VI). Timing is measured using `time.perf_counter_ns()` (nanosecond resolution). Power samples are collected at 1 kHz throughout the session and integrated into per-suite energy values. MAVLink integrity is monitored via a dedicated sniff port, tracking per-message-type counts, sequence gaps, heartbeat intervals, and RTT via COMMAND_ACK and active PING probes.

TABLE IV: KEM Primitive Timing on RPi4 (avg over all suites using each KEM, no-ddos baseline)

Algorithm	KeyGen (ms)	Encaps (ms)	Decaps (ms)	HS (ms)
ML-KEM-512	0.62	0.33	0.16	233.5
ML-KEM-768	0.50	0.36	0.18	712.4
ML-KEM-1024	0.28	0.37	0.24	403.7
HQC-128	2.39	45.91	7.26	276.1
HQC-192	4.39	138.49	18.25	858.8
HQC-256	8.44	252.86	29.73	678.5
McEliece-348864	122.74	0.88	21.06	570.7
McEliece-460896	160.86	3.28	59.90	9196.5
McEliece-8192128	502.89	3.35	146.77	2228.0

VI. Results and Analysis

We present results from the complete 72-suite benchmark across all three operational phases measured on the Raspberry Pi 4 platform. All numbers are single-run observations from the production benchmark (run tag `20260220_full`), measured under controlled lab conditions with CPU governor locked to `performance` mode.

A. Individual KEM Benchmarks

Table IV reports the average KEM primitive timing across all suites using each KEM algorithm.

ML-KEM primitives are one to three orders of magnitude faster than HQC and Classic McEliece. ML-KEM-768 key generation completes in 0.50 ms, while Classic-McEliece-8192128 requires 502.89 ms—a factor of $\sim$1000$\times$. The high average handshake times for ML-KEM-768 (712.4 ms) compared to ML-KEM-512 (233.5 ms) reflect that ML-KEM-768 is paired exclusively with L3 signatures (ML-DSA-65, SPHINCS$^+$-192s) which are heavier than L1 signatures; individual ML-KEM-768 + ML-DSA-65 handshakes complete in only 21.3 ms.

HQC encapsulation is notably expensive: HQC-256 requires 252.86 ms per encapsulation, making it the costliest per-handshake KEM operation. Classic McEliece's key generation dominates its handshake cost, while its encapsulation is fast (leveraging the Niederreiter dual encoding).

B. Signature Benchmarks

Table V reports signature timing averaged over all suites pairing each signature.

Falcon-512 achieves the fastest signing (0.50 ms) and produces the smallest signatures (656 bytes), making it attractive for UAV deployment. ML-DSA offers slightly larger signatures but avoids the floating-point sampling complexity of Falcon. SPHINCS$^+$ signatures are one to two orders of magnitude larger (up to 29 792 bytes for SPHINCS$^+$-256s)—a cost that dominates handshake time for suites pairing SPHINCS$^+$ with any KEM.

TABLE V: Signature Primitive Timing on RPi4 (no-ddos baseline)

Algorithm	Sign (ms)	Verify (ms)	Sig Size (B)	HS (ms)
ML-DSA-44	0.84	1.49	2 420	131.0
ML-DSA-65	0.87	2.78	3 309	327.6
ML-DSA-87	2.01	5.62	4 627	843.2
Falcon-512	0.50	1.47	656	137.0
Falcon-1024	2.29	6.58	1 271	666.1
SPHINCS$^+$-128s	—	—	7 856	—
SPHINCS$^+$-192s	—	—	16 224	—
SPHINCS$^+$-256s	—	—	29 792	—

TABLE VI: AEAD Per-Packet Cost on RPi4 (averaged over all 72 suites, no-ddos)

AEAD	Encrypt (ns)	Decrypt (ns)	Power (W)
ChaCha20-Poly1305	71 441	82 504	3.92
AES-256-GCM	76 341	83 982	3.94
Ascon-128a	1 374 916	999 440	4.13

C. AEAD Data-Plane Cost

Table VI reports the per-packet AEAD cost for a typical MAVLink payload ($\approx$280 B).

ChaCha20-Poly1305 and AES-256-GCM are within 7% of each other. On the Cortex-A72 without hardware AES-NI, ChaCha20 enjoys a marginal advantage due to its register-friendly ARX operations. Ascon-128a is $\sim$19$\times$ slower in encryption and $\sim$12$\times$ slower in decryption, reflecting its pure-Python software implementation (the C native extension requires separate compilation for `aarch64`). The 0.21 W power premium of Ascon-128a over ChaCha20 is significant on a 4 W platform.

D. Suite-Level Handshake Results

Table VII presents handshake latency for selected suites across all three NIST levels.

The fastest suite (ML-KEM-512 + Falcon-512 + ChaCha20-Poly1305) completes a full PQC handshake in 11.1 ms—well within MAVLink heartbeat deadlines. Even at NIST Level 5, ML-KEM-1024 + ML-DSA-87 achieves 21.4 ms, demonstrating that lattice-based PQC is practical for real-time UAV communication.

The slowest suite (Classic-McEliece-460896 + SPHINCS$^+$-192s + AES-GCM) requires 48.3 s—dominated by McEliece's 524 KB public key transfer and SPHINCS$^+$'s 16 KB signature size, causing network transmission delays that dwarf the cryptographic computation.

E. Rekey Overhead Analysis

Using Equation 9, we compute the rekey overhead fraction for representative suites at the default 60 s rekey interval:

ML-KEM-based suites incur negligible rekey overhead ($<$ 0.04%), enabling frequent rekeying for forward secrecy. Heavy suites (Classic McEliece) should use longer rekey intervals or be reserved for initial establishment only.

TABLE VII: Selected Suite Handshake Latency (no-ddos, RPi4)

Suite	Level	HS (ms)
ML-KEM-512 + Falcon-512 + ChaCha20	L1	11.1
ML-KEM-512 + ML-DSA-44 + AES-GCM	L1	15.1
ML-KEM-768 + ML-DSA-65 + AES-GCM	L3	21.3
ML-KEM-1024 + ML-DSA-87 + AES-GCM	L5	21.4
HQC-128 + ML-DSA-44 + AES-GCM	L1	61.4
HQC-256 + ML-DSA-87 + AES-GCM	L5	274.5
McEliece-348864 + Falcon-512 + AES-GCM	L1	376.4
McEliece-8192128 + ML-DSA-87 + AES-GCM	L5	2 602.0
McEliece-460896 + SPHINCS$^+$-192s + AES-GCM	L3	48 257.1

TABLE VIII: Rekey Overhead at 60 s Interval

Suite	Φ(60 s)
ML-KEM-512 + Falcon-512 + ChaCha20	0.018%
ML-KEM-768 + ML-DSA-65 + AES-GCM	0.035%
HQC-256 + ML-DSA-87 + AES-GCM	0.456%
McEliece-8192128 + ML-DSA-87 + AES-GCM	4.156%

F. DDoS Impact on PQC Performance

Table IX compares the reference suite across baseline, XGBoost, and TST phases.

The XGBoost detector raises the CPU temperature by 3 °C but does not measurably increase power for this suite. The TST detector adds 0.05 W and 3 °C to the steady state. Handshake times *decrease* slightly in detector phases likely due to CPU cache warming from the sustained detector computation.

The additional power consumption of the detectors—measured at 0.95 W for XGBoost and 1.97 W for TST in isolation—is the primary system-level cost. This overhead is constant regardless of the PQC suite, making detector management the dominant factor in the scheduler's energy budget.

G. Power and Thermal Behaviour

Average power consumption is remarkably stable across KEM families (3.96 W to 4.01 W). The primary power differentiator is AEAD selection (3.92 W for ChaCha20 vs. 4.13 W for Ascon-128a) and detector presence. Classic McEliece shows higher energy per suite (45.8 J vs. 37.4 J for ML-KEM) due to the longer active time during key generation, despite similar instantaneous power.

H. Scheduler Decision Impact

The MDEAS scheduler responds to changing conditions along its three axes. Table XI illustrates representative decisions.

The AEAD preference order on ARM Cortex-A72 is ChaCha20 $>$ AES-GCM $>$ Ascon-128a, reflecting the lack of hardware AES acceleration and the software overhead of Ascon. The scheduler's seeded AEAD cost profiles (Table XII)

TABLE IX: DDoS Detector Impact on ML-KEM-768 + ML-DSA-65 + AES-GCM

Phase	HS (ms)	Power (W)	Temp (°C)	Energy (J)
Baseline	21.3	3.91	61.3	37.9
XGBoost	19.6	3.91	64.3	37.9
TST	16.7	3.96	64.3	38.4

TABLE X: Power and Temperature Summary Across KEM Families (no-ddos)

KEM Family	Power (W)	σ (W)	Temp (°C)	Energy (J)
ML-KEM	3.99	0.04	62.8	37.4
HQC	4.01	0.04	62.3	37.8
Classic McEliece	3.96	0.08	60.9	45.8
Overall: min 3.76 W, *max* 4.19 W, *avg* 3.99 W				

TABLE XI: MDEAS Scheduler Decision Scenarios

Condition	Action
Temp $> 80\,^{\circ}\mathrm{C}$ or Battery $< 14\,\mathrm{V}$	EMERGENCY: L1 + cheapest AEAD, kill detector
Temp crossing $70\,^{\circ}\mathrm{C}$ within $60\,\mathrm{s}$	SWITCH_AEAD to cooler cipher
DDoS detected (drop ratio > 0.10)	EMERGENCY: cheapest AEAD, lock
Detector thermal ceiling exceeded	DOWNGRADE_DETECTOR
Link gap $p_{95} > 1\,\mathrm{s}$	DOWNGRADE_LEVEL
Stable for $30\,\mathrm{s}$ (armed: $90\,\mathrm{s}$)	UPGRADE_LEVEL

TABLE XII: AEAD Benchmark-Seeded Cost Profiles

AEAD	Enc (ns)	Dec (ns)	ΔT (°C)
ChaCha20-Poly1305	63 500	70 700	0.0
AES-256-GCM	66 900	73 600	−0.1
Ascon-128a	1 327 100	960 500	+1.5

drive switching decisions without requiring a learning warm-up:

I. Graceful Degradation

The scheduler implements graceful degradation through a tiered response. When thermal stress is detected, the system first switches AEAD (Axis 1)—the cheapest corrective action affecting per-packet cost. If stress persists, it downgrades the NIST level (Axis 2), reducing handshake complexity at the cost of security margin. Finally, under sustained overload, it sheds the DDoS detector (Axis 3), freeing the largest block of compute resources (0.95 –-$1.97\,\mathrm{W}$).

This layered approach ensures that *connectivity is always maintained*: the system never drops to unencrypted operation, but gracefully trades security margin and detection capability for thermal and energy sustainability.

VII. Discussion

Why ML-KEM Dominates. ML-KEM (Kyber) achieves sub-millisecond primitive operations across all three security levels [5], with compact key and ciphertext sizes. Combined with ML-DSA [6], the fastest L3 suite completes a full PQC handshake in $21.3\,\mathrm{ms}$—two orders of magnitude below the MAVLink heartbeat interval ($1\,\mathrm{s}$). This makes ML-KEM the clear first choice for UAV deployment.

Why Classic McEliece Is Costly. Classic McEliece's security rests on decades-old coding theory [10], but its public key sizes (261 KB to 1.3 MB) create network transmission bottlenecks that dominate handshake latency. The 48-second handshake for McEliece-460896 + SPHINCS$^+$-192s is unacceptable for real-time UAV operation. However, McEliece offers a conservative, structurally distinct security assumption and may serve as a *fallback* KEM for infrequent key establishment in high-security scenarios.

Why SPHINCS$^+$ Is Heavy. SPHINCS$^+$ (SLH-DSA) [8] provides hash-based signatures with minimal cryptographic assumptions, but its signatures (7–30 KB) increase both wire overhead and verification time. Suites combining SPHINCS$^+$ with any KEM are dominated by the signature cost, as reflected in the per-signature average handshake times exceeding the per-KEM averages.

Why AEAD Choice Is Mostly Negligible. At 71 –-$84\,\mu\mathrm{s}$ per packet, ChaCha20 and AES-GCM contribute less than $0.1\,\mathrm{ms}$ per MAVLink message. The exception is Ascon-128a, which at $1.37\,\mathrm{ms}$ per encryption approaches the MAVLink inter-packet interval during high telemetry rates. This cost difference drives the scheduler's AEAD-axis decisions and motivates future hardware-accelerated Ascon implementations.

Why Detector Energy Dominates. The XGBoost detector adds $0.95\,\mathrm{W}$ (24% of baseline) and TST adds $1.97\,\mathrm{W}$ (49% of baseline)—costs that exceed the entire variation across all 72 PQC suites. This asymmetry means the scheduler's Axis 3 (detector level) has the largest single lever for energy management, justifying its prominent position in the decision cascade.

Limitations. Our evaluation uses a single Raspberry Pi 4 under controlled lab conditions. Field deployment introduces additional variables: vibration, ambient temperature, battery depletion curves, and multi-path WiFi interference. The benchmark runs each suite for $10\,\mathrm{s}$, which captures steady-state costs but may not fully characterise long-running thermal dynamics. The SPHINCS$^+$ suite timing includes network transfer of the large signature, which would vary with link bandwidth.

VIII. Conclusion and Future Work

We presented SecureTunnel, a comprehensive post-quantum cryptographic tunnel for MAVLink-based UAV communication. The system constructs 72 cryptographic suites from the NIST-level-consistent product of 9 KEMs, 8 digital signatures, and 3 AEAD ciphers. A two-message handshake protocol achieves mutual authentication, forward secrecy, and anti-downgrade protection.

Our full-system benchmark on a Raspberry Pi 4 companion computer demonstrates that lattice-based PQC (ML-KEM +

ML-DSA) is immediately deployable, with handshakes completing in 11.1 –-$21.3\,\mathrm{ms}$ at levels L1 through L5. HQC provides a code-based alternative at moderate cost (61 –-$275\,\mathrm{ms}$). Classic McEliece remains viable for infrequent key establishment but is prohibitive for frequent rekeying due to large public keys.

The MDEAS scheduler jointly optimises three axes—AEAD selection, NIST level, and DDoS detector intensity—using real-time telemetry, achieving graceful degradation without disconnection. The dominant energy lever is detector management (0.95 –-$1.97\,\mathrm{W}$), exceeding the variance across all PQC cipher suites.

Optimal configurations:

- **General deployment:** ML-KEM-768 + ML-DSA-65 + ChaCha20-Poly1305 (L3, $21.3\,\mathrm{ms}$, $3.91\,\mathrm{W}$).
- **Minimum latency:** ML-KEM-512 + Falcon-512 + ChaCha20-Poly1305 (L1, $11.1\,\mathrm{ms}$).
- **Maximum security:** ML-KEM-1024 + ML-DSA-87 + AES-256-GCM (L5, $21.4\,\mathrm{ms}$).
- **Conservative:** Classic-McEliece-348864 + Falcon-512 + AES-GCM (L1, structurally diverse, $376\,\mathrm{ms}$).

Future Work. Several directions emerge: (1) *Reinforcement Learning Scheduling*: replacing the rule-based MDEAS with an RL agent that learns optimal policies from flight data. (2) *Hardware-Accelerated Ascon*: leveraging dedicated Ascon hardware [11] to close the $19\times$ software gap with AES-GCM. (3) *Multi-Drone Swarms*: extending the tunnel to mesh topologies with group key management. (4) *Formal Verification*: proving the handshake protocol correct using symbolic model checking (e.g., ProVerif). (5) *Hybrid PQC-Classical Modes*: combining classical ECDH with PQC KEM for defense-in-depth during the transition period.

References

[1] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," NIST, Gaithersburg, MD, Aug. 2024.

[2] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)," NIST, Gaithersburg, MD, Aug. 2024.

[3] National Institute of Standards and Technology, "FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)," NIST, Gaithersburg, MD, Aug. 2024.

[4] National Institute of Standards and Technology, "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," NIST IR 8545, 2024.

[5] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367.

[6] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Trans. Cryptographic Hardware and Embedded Systems (TCHES)*, vol. 2018, no. 1, pp. 238–268, 2018.

[7] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-Fourier lattice-based compact signatures over NTRU," NIST PQC Round 3 Submission, 2020.

[8] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS$^+$ signature framework," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 2129–2146.

[9] C. Aguilar Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, E. Persichetti, G. Zémor, and I. C. Bos, "Hamming Quasi-Cyclic (HQC)," NIST PQC Round 4 Submission, 2023.

[10] D. J. Bernstein, T. Chou, T. Lange, I. von Maurich, R. Misoczki, R. Niederhagen, E. Persichetti, C. Peters, P. Schwabe, N. Sendrier, J. Szefer, and W. Wang, "Classic McEliece: Conservative code-based cryptography," NIST PQC Round 4 Submission, 2023.

[11] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *J. Cryptology*, vol. 34, no. 3, 2021.

[12] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, Internet Engineering Task Force, May 2010.

[13] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, Internet Engineering Task Force, Jun. 2018.

[14] J. Salowey, A. Choudhury, and D. McGrew, "AES Galois Counter Mode (GCM) cipher suites for TLS," RFC 5288, Internet Engineering Task Force, Aug. 2008.

[15] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Internet Engineering Task Force, Feb. 1997.

[16] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1994, pp. 124–134.

[17] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.

[18] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[19] D. Stebila and M. Mosca, "Open Quantum Safe," https://openquantumsafe.org, 2024. [Online; accessed Feb. 2026].

[20] Open Quantum Safe Project, "liboqs-python: Python 3 wrapper for liboqs," https://github.com/open-quantum-safe/liboqs-python, 2024.

[21] Python Cryptographic Authority, "cryptography: Cryptographic recipes and primitives for Python," https://cryptography.io, 2024.

[22] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4," *IACR Cryptology ePrint Archive*, Report 2019/844, 2019.

[23] F. Schmid, T. Unterluggauer, and S. Mangard, "Post-quantum cryptography on ARM Cortex-A: Benchmarking NIST PQC candidates," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, 2023, pp. 1–6.

[24] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Proc. Post-Quantum Cryptography (PQCrypto)*, Springer LNCS, 2020, pp. 72–91.

[25] W. Horn, K. Kanning, and B. Lipp, "Rosenpass: Post-quantum secure VPN," https://rosenpass.eu, 2023.

[26] A. Hülsing, K.-K. Ning, P. Schwabe, and F. Weber, "Post-quantum WireGuard: Practical post-quantum VPN," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 304–321.

[27] R. Altawy and A. M. Youssef, "Security, privacy, and safety aspects of civilian drones: A survey," *ACM Trans. Cyber-Physical Systems*, vol. 1, no. 2, pp. 1–25, 2017.

[28] N. K. Pham and M. Vakilinia, "Post-quantum cryptography analysis for UAV communication systems," in *Proc. IEEE Int. Conf. on Communications (ICC)*, 2023, pp. 1–6.

[29] MAVLink Developer Team, "MAVLink protocol specification v2.0," https://mavlink.io/en/, 2024.

[30] ArduPilot Developer Team, "MAVProxy: A UAV ground station software package for MAVLink-based systems," https://ardupilot.github.io/MAVProxy/, 2024.

[31] ArduPilot Developer Team, "ArduPilot: Versatile, trusted, open autopilot software," https://ardupilot.org, 2024.

[32] Texas Instruments, "INA219: Bidirectional current/power monitor with I^{2}C interface," Datasheet SBOS448G, 2024.

[33] Holybro, "Pixhawk 6C Mini: Open-source autopilot for drones," https://holybro.com/products/pixhawk-6c-mini, 2024.

[34] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[35] H. Zhang, L. Yu, and X. Chen, "Transformer-based network intrusion detection with temporal attention," *IEEE Trans. Information Forensics and Security*, vol. 18, pp. 2035–2049, 2023.
[36] E. Barker and A. Roginsky, "Transitioning the use of cryptographic algorithms and key lengths," NIST SP 800-131A Rev. 2, Mar. 2019.